Lecture8 — JSON in JavaScript & String Methods

JSON in JavaScript

JSON (JavaScript Object Notation) is a lightweight text format for storing and exchanging data.

Key rules

- Data is written as **objects** and **arrays**.
- **Keys must be in double quotes**: `"name"`
- Valid value types: string, number, object, array, boolean, `null`.
- JSON is **data only** (no functions, `undefined`, comments, or trailing commas).

JSON example

```
// This is a JavaScript object (not JSON yet)
const person = {
    name: "Aya",
    age: 20,
    skills: ["JS", "HTML"],
    isStudent: true,
    nickname: null
};

// Convert JS object -> JSON string
const jsonString = JSON.stringify(person);
console.log(jsonString);

// Convert JSON string -> JS object
```

```
const parsed = JSON.parse(jsonString);
console.log(parsed.name);
```

When to use

- Sending data to APIs.
- Saving data in files or localStorage.
- Receiving data from servers.

💡 If you see JSON in a file, it usually looks like this:

```
{
    "name": "Aya",
    "age": 20,
    "skills": ["JS", "HTML"],
    "isStudent": true,
    "nickname": null
}
```

JSON data types

number, string, array, {}, null, boolean

The date is in a string datatype, undefined isn't allowed in json data type, function in string, comment aren't allowed

data type of array is object

error: type of string =⇒ json

```
// JSON string
const jsonString = '{"name": "John", "age": 30, "city": "New York"}';

// Convert to JavaScript object
```

```
const obj = JSON.parse(jsonString);
console.log(obj); // {name: "John", age: 30, city: "New Yor
k"}
console.log(obj.name); // "John"
```

String methods

JavaScript provides many useful string methods for working with text data. Here are the most commonly used ones:

Basic Access and Length

- `length` - Returns the length of the string
- `charAt(index)` - Returns the character at the specified index
- `charCodeAt(index)` - Returns the Unicode value of the character at the specified index
- `indexOf(searchValue, fromIndex)` - Returns the first occurrence of a substring
- `lastIndexOf(searchValue, fromIndex)` - Returns the last occurrence of a substring
- `string.split(separator, limit))` - Returns the last occurrence of a substring

```
let str = "Hello World";
console.log(str.length); // 11
console.log(str.charAt(4)); // "o"
console.log(str.indexOf("World")); // 6
```

```
let str = "apple,banana,orange";
let arr = str.split(",");
console.log(arr); // ["apple", "banana", "orange"]

let str = "Hello";
let arr = str.split("");
console.log(arr); // ["H", "e", "l", "l", "o"]
```

```
let str = "Hello World";
let arr = str.split(" ");
console.log(arr); // ["Hello", "World"]

let text = "   Hello World!   ";
let result = text.trim();

console.log(result);
// Output: "Hello World!"
```

Substring Extraction

- `slice(start, end)` - Extracts a section of a string and returns it as a new string
- `substring(start, end)` - Similar to slice but doesn't accept negative indices
- `substr(start, length)` - Extracts characters from start position for specified length

```
let str = "JavaScript";
console.log(str.slice(0, 4)); // "Java"
console.log(str.substring(4, 9)); // "Scrip"
console.log(str.substr(4, 5)); // "Scri"
```

Case Conversion

- `toLowerCase()` - Converts to lowercase
- `toUpperCase()` - Converts to uppercase
- `toLocaleCase()` - Converts to locale-specific case

```
let str = "Hello World";
console.log(str.toLowerCase()); // "hello world"
console.log(str.toUpperCase()); // "HELLO WORLD"
```

Search and Pattern Matching

- `includes(searchValue, fromIndex)` - Checks if string contains the substring
- `startsWith(searchValue, fromIndex)` - Checks if string starts with the substring
- `endsWith(searchValue, length)` - Checks if string ends with the substring
- `match(regexp)` - Matches against a regular expression
- `search(regexp)` - Searches for a match against a regular expression

```
let str = "Hello World";
console.log(str.includes("World")); // true
console.log(str.startsWith("Hello")); // true
console.log(str.endsWith("World")); // true
console.log(str.match(/l/g)); // ["l", "l", "l"]
```

Modification and Replacement

- `replace(searchValue, replaceValue)` - Replaces matched patterns
- `replaceAll(searchValue, replaceValue)` - Replaces all occurrences
- `split(separator, limit)` - Splits string into an array

```
let str = "Hello World, Hello Universe";
console.log(str.replace("Hello", "Hi")); // "Hi World, Hello
Universe"
console.log(str.replaceAll("Hello", "Hi")); // "Hi World, Hi
Universe"
console.log(str.split(" ")); // ["Hello", "World,", "Hello",
"Universe"]
```

Trimming and Padding

- `trim()` - Removes whitespace from both ends
- `trimStart()` - Removes whitespace from the beginning

- `trimEnd()` - Removes whitespace from the end
- `padStart(targetLength, padString)` - Pads the string from the start
- `padEnd(targetLength, padString)` - Pads the string from the end

```
let str = "   Hello World   ";
console.log(str.trim()); // "Hello World"
console.log(str.trimStart()); // "Hello World   "
console.log("5".padStart(3, "0")); // "005"
console.log("5".padEnd(5, "*")); // "5****"
```

Comparison Methods

- `localeCompare(compareString)` - Compares two strings in locale-specific order
- `repeat(count)` - Returns the string repeated count times

```
let str1 = "apple";
let str2 = "banana";
console.log(str1.localeCompare(str2)); // -1 (apple comes bef
ore banana)
console.log("a".repeat(3)); // "aaa"
```

Static String Methods

- `String.fromCharCode()` - Creates a string from Unicode values
- `String.fromCodePoint()` - Creates a string from code points
- `String.raw` - Template literal tag that doesn't escape backslashes

```
console.log(String.fromCharCode(72, 101, 108, 108, 111)); //
"Hello"
console.log(String.fromCodePoint(128512)); // "😀"
```

These methods provide comprehensive functionality for working with strings in JavaScript, from basic manipulation to complex pattern matching and formatting.